



RELATÓRIO TÉCNICO

Atividade 4: Multiplexador Genérico

ALUNOS

Aluno(a): Tiago Navarro Bettanin
Aluno(a): João Antonio Sitta Martins

R.A: 2477866
R.A: 2553058

1 Introdução

Nesta atividade, o objetivo foi desenvolver, em VHDL, um **multiplexador genérico**, ou seja, um circuito de seleção em que tanto o número de entradas quanto a quantidade de bits por entrada podem ser definidos de forma arbitrária no momento da instanciación do componente. O multiplexador é um bloco combinacional clássico: dadas N entradas de M bits cada, e um sinal de seleção S , apenas uma das entradas é encaminhada para a saída.

O conceito novo introduzido nesta atividade é o uso da estrutura **generic** da linguagem VHDL. Ela permite parametrizar a entidade, tornando o código reutilizável: ao invés de escrever um multiplexador específico para 4 entradas de 2 bits, outro para 8 entradas de 4 bits, e assim por diante, descreve-se um único componente cujos parâmetros são definidos no momento da síntese ou simulação. Conforme orientado, optou-se por parametrizar o circuito a partir do **número de bits de seleção (S)** e do **número de bits por entrada (M)**, obtendo o número de entradas pela relação $N = 2^S$. Dessa forma, basta alterar dois inteiros para gerar multiplexadores de qualquer dimensão.

Outro recurso explorado foi o **slicing dinâmico** de um `bit_vector`, em que o vetor de entrada concentra todas as N palavras de M bits em um único barramento de $N \cdot M$ bits, e a saída é obtida selecionando a fatia correspondente ao valor de `selecao`.

2 Desenvolvimento

2.1 Descrição da Lógica Implementada

A entidade possui dois parâmetros genéricos: **S**, que define o número de bits de seleção, e **M**, que define o número de bits por entrada. O número total de entradas é obtido por 2^S , e o barramento de entrada **N** possui largura $2^S \cdot M$ bits, concatenando todas as palavras de entrada em um único vetor.

A entrada de seleção foi declarada como um **integer** no intervalo de 0 a $2^S - 1$, o que facilita o cálculo dos índices de fatiamento. A saída **z** possui M bits e é atribuída diretamente pela operação:

$$z \leftarrow N[(selecao + 1) \cdot M - 1 \text{ downto } selecao \cdot M]$$

Ou seja, quando `selecao = 0`, a saída recebe os bits de $M - 1$ a 0 (a primeira palavra); quando `selecao = 1`, recebe os bits de $2M - 1$ a M (a segunda palavra), e assim por diante. Essa abordagem evita o uso de longas estruturas `when/else` ou `with select`, deixando o código compacto e totalmente parametrizável.

2.2 Simulação Funcional

Para validar o circuito, foi realizada uma simulação com $S = 4$ e $M = 8$, resultando em $2^4 = 16$ entradas de 8 bits cada (totalizando 128 bits no barramento N), conforme sugerido no enunciado. Os valores foram apresentados em hexadecimal para facilitar a leitura.

O vetor de entrada utilizado foi `N = E53B30287500C8518F6CFDD7C69DFB32`, que, dividido em palavras de 8 bits a partir do bit menos significativo, corresponde às 16 entradas listadas na Tabela 1.

Tabela 1: Mapeamento entre o índice de seleção e a respectiva palavra de 8 bits.

selecao	Valor (hex)	selecao	Valor (hex)
0 (0000)	32	8 (1000)	51
1 (0001)	FB	9 (1001)	C8
2 (0010)	9D	10 (1010)	00
3 (0011)	C6	11 (1011)	75
4 (0100)	D7	12 (1100)	38
5 (0101)	FD	13 (1101)	30
6 (0110)	6C	14 (1110)	3B
7 (0111)	8F	15 (1111)	E5

A Figura 1 apresenta o waveform da simulação. À medida que `selecao` percorre os valores de 0000 a 1111, a saída `z` reproduz exatamente a sequência 32, FB, 9D, C6, D7, FD, 6C, 8F, 51, C8, 00, 75, 38, 30, 3B, E5, comprovando o correto funcionamento do multiplexador genérico para uma configuração de grande porte.

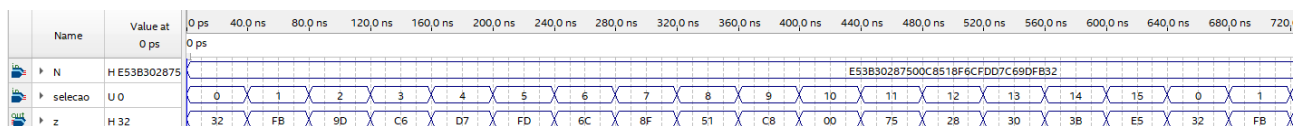


Figura 1: Simulação do multiplexador genérico para $S = 4$ e $M = 8$ (16 entradas de 8 bits, valores em hexadecimal).

2.3 Diagrama RTL

A síntese no Quartus Prime gera o diagrama RTL do circuito, permitindo verificar a estrutura lógica gerada a partir do código genérico. A Figura 2 apresenta o diagrama RTL para a configuração utilizada na simulação ($S = 4$, $M = 8$), em que é possível observar os 8 blocos multiplexadores internos, um para cada bit da saída, e a forma como o barramento de 128 bits é distribuído entre as 16 entradas selecionáveis.

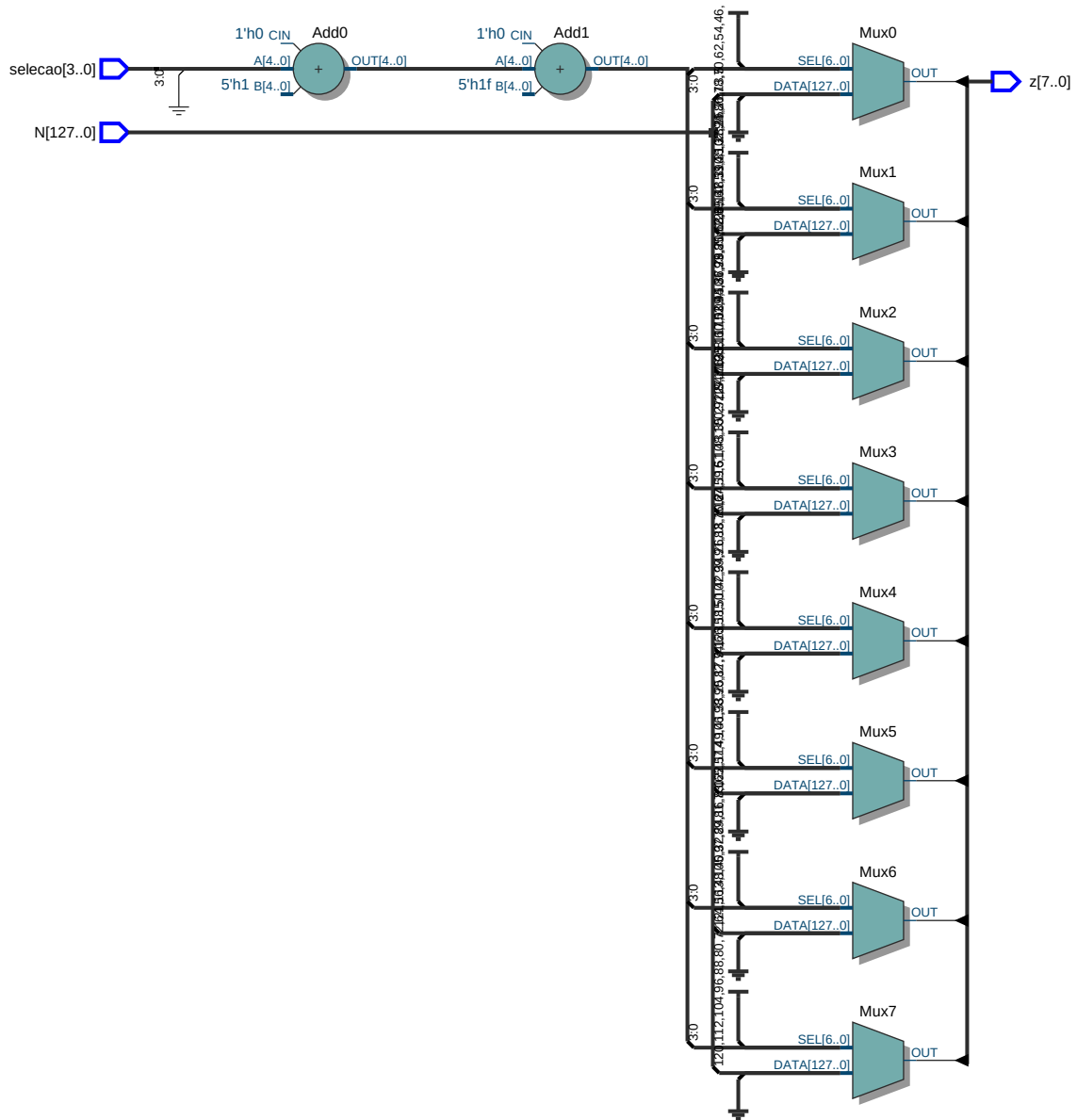


Figura 2: Diagrama RTL para $S = 4$ e $M = 8$ (16 entradas de 8 bits).

Já a Figura 3 apresenta o diagrama RTL da versão implementada fisicamente na placa, com $S = 2$ e $M = 2$ (4 entradas de 2 bits cada).

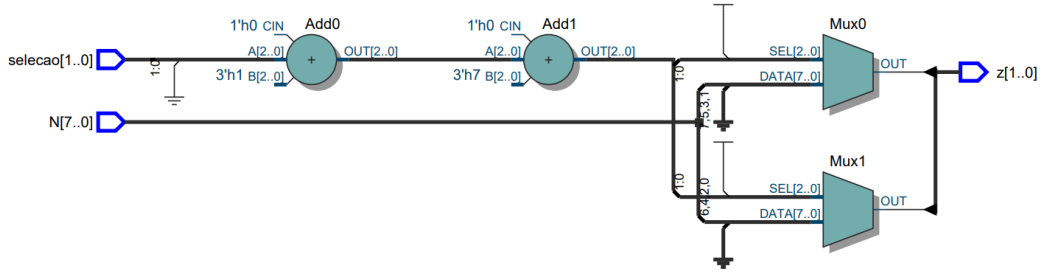


Figura 3: Diagrama RTL para $S = 2$ e $M = 2$ (4 entradas de 2 bits), versão implementada na placa.

2.4 Implementação na Placa DE10-Lite

Para a implementação física, foi utilizada a configuração mínima exigida pelo enunciado: $S = 2$ e $M = 2$, ou seja, 4 entradas de 2 bits cada, totalizando 8 bits no barramento de entrada. As 8 chaves da placa foram associadas aos bits do barramento N, outras 2 chaves foram utilizadas para o sinal de seleção, e a saída z (2 bits) foi conectada a LEDs.

2.4.1 Atribuição dos Pinos (Pin Planner)

Para que o circuito sintetizado interaja com os componentes físicos da placa DE10-Lite, é necessário associar cada porta da entidade VHDL a um pino específico do FPGA. Essa associação é feita pelo Pin Planner do Quartus Prime, e a Figura 4 apresenta a configuração utilizada nesta atividade.

Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard	Reserved	Current Strength	Slew Rate	Differential Pair	Strict Preservation
N[7]	Input	PIN_D12	7	B7_NO	PIN_D12	2.5 V		12mA (default)			
N[6]	Input	PIN_C12	7	B7_NO	PIN_C12	2.5 V		12mA (default)			
N[5]	Input	PIN_A12	7	B7_NO	PIN_A12	2.5 V		12mA (default)			
N[4]	Input	PIN_B12	7	B7_NO	PIN_B12	2.5 V		12mA (default)			
N[3]	Input	PIN_A13	7	B7_NO	PIN_A13	2.5 V		12mA (default)			
N[2]	Input	PIN_A14	7	B7_NO	PIN_A14	2.5 V		12mA (default)			
N[1]	Input	PIN_B14	7	B7_NO	PIN_B14	2.5 V		12mA (default)			
N[0]	Input	PIN_F15	7	B7_NO	PIN_F15	2.5 V		12mA (default)			
secao[1]	Input	PIN_C11	7	B7_NO	PIN_C11	2.5 V		12mA (default)			
secao[0]	Input	PIN_C10	7	B7_NO	PIN_C10	2.5 V		12mA (default)			
z[1]	Output	PIN_A11	7	B7_NO	PIN_A11	2.5 V		12mA (default)	2 (default)		
z[0]	Output	PIN_B11	7	B7_NO	PIN_B11	2.5 V		12mA (default)	2 (default)		
<<new node>>											

Figura 4: Listagem do Pin Planner com as atribuições das entradas e saídas.

Os 8 bits do barramento N[7..0] foram conectados às chaves deslizantes (SW) da placa, distribuídas nos pinos PIN_D12, PIN_C12, PIN_A12, PIN_B12, PIN_A13, PIN_A14, PIN_B14 e PIN_F15 (as 8 primeiras chaves da esquerda para a direita). Esses 8 bits representam, na prática, as 4 entradas de 2 bits do multiplexador, agrupadas no mesmo barramento.

Os 2 bits de seleção, secao[1..0], foram associados às chaves localizadas nos pinos PIN_C11 e PIN_C10 (as duas últimas chaves da direita). Já a saída z[1..0] foi conectada aos LEDs nos pinos PIN_A11 e PIN_B11 (os dois primeiros LEDs), permitindo a visualização direta da palavra selecionada.

2.4.2 Foto da Placa em Operação

A Figura 5 mostra a placa DE10-Lite em operação, com as chaves selecionando uma das entradas e os LEDs exibindo o valor correspondente.



Figura 5: Placa DE10-Lite executando o multiplexador genérico de 4 entradas de 2 bits.

2.5 Código VHDL

O código abaixo implementa o multiplexador genérico. Os comentários explicam o papel de cada parâmetro e da expressão de fatiamento que realiza a seleção da entrada.

```

1  library ieee ;
2  use ieee.std_logic_1164.all ;
3  -----
4  entity projeto is
5      -- Parametros genericos:
6      --   S = numero de bits de selecao  (numero de entradas N = 2^S)
7      --   M = numero de bits por entrada
8      generic (
9          S: integer := 2;
10         M: integer := 2
11     ) ;
12     port (
13         -- Barramento de entrada com todas as N palavras concatenadas
14         -- Largura total = 2^S * M bits
15         N: in bit_vector((2**S * M) - 1 downto 0);
16         -- Sinal de selecao como inteiro de 0 a 2^S - 1
17         selecao: in integer range 0 to (2**S)-1;
18         -- Saida com a palavra selecionada (M bits)
19         z: out bit_vector(M - 1 downto 0)
20     );
21 end entity ;
22 -----
23 architecture projeto of projeto is
24 begin
25     -- Slicing dinamico: seleciona a fatia de M bits correspondente
26     -- a entrada indicada por 'selecao'.
27     -- Quando selecao=0 -> bits [M-1 downto 0]
28     -- Quando selecao=1 -> bits [2M-1 downto M]
29     -- e assim por diante.
30     z <= N( ((selecao + 1) * M) - 1 downto (selecao * M) );
31 end architecture;

```

3 Conclusão

A atividade permitiu consolidar o uso da estrutura `generic` em VHDL, recurso essencial para a descrição de componentes parametrizáveis e reutilizáveis. Foi possível observar, na prática, como um único código pode gerar circuitos de dimensões muito distintas, desde um multiplexador modesto de 4 entradas de 2 bits, sintetizado na placa DE10-Lite, até um multiplexador de 16 entradas de 8 bits, validado por simulação, bastando alterar os valores dos parâmetros `S` e `M`.

Também foi explorado o uso de `integer` em portas de seleção e o fatiamento dinâmico de `bit_vector`, técnica que substitui de forma elegante longas listas de `when/else`. A simulação em hexadecimal e a comparação dos valores das fatias com a saída comprovaram o funcionamento correto do circuito, e a inspeção do diagrama RTL evidenciou a estrutura efetivamente sintetizada pelo Quartus Prime. Com isso, os requisitos propostos para a atividade foram integralmente atendidos [1].

Referências

- [1] Atividade 4: Multiplexador genérico. Documento de enunciado fornecido pelo professor, disciplina de Lógica Reconfigurável, UTFPR Campus Apucarana, 2026/1.